



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09-2

NOMBRE COMPLETO: RIVERA ESPINOZA JOSE LUIS

N° de Cuenta: 318152525

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 27 DE OCTUBRE DE 2024

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

- Para el primer parte del ejercicio, era hacer que, en el cartel del arco, el desplazamiento de una textura con la tipografía de nuestro universo que debía decir “Proyecto CGEIHHC MONOPOLY”.

```
//Arco
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Arco_M.RenderModel();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Letrero_M.RenderModel();

toffsetflechau += 0.00001;
toffsetflechav = 0.000;
//para que no se desborde la variable
if (toffsetflechau > 1.0)
    toffsetflechau = 0.0;
toffset = glm::vec2(toffsetflechau, toffsetflechav);

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.08f, 2.0f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f, 0.5f, 0.5f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Letras.UseTexture();
meshList[4]->RenderMesh();
```



- Para la parte dos, teníamos que separar las cabezas del dragón y agregarle animaciones como, que el dragón hiciese un recorrido de ida y vuelta, así como el aleteo de las alas, y el movimiento de las cabezas, adicionalmente agregarle texturas que hiciesen representación a algún elemento.

```

angulovaria += 5.0f*deltaTime;
ejexvaria += 0.05*deltaTime;

```

```

//cabeza 1
varial += 5.0f * deltaTime;
//cabeza 2
varia2 += 5.0f * deltaTime;
//cabeza 3
varia3 += 5.0f * deltaTime;
//cabeza 4
varia4 += 1.0f * deltaTime;
//cabeza 5
varia5 += 5.0f * deltaTime;

```

```

model = modelaux2;
model = glm::translate(model, glm::vec3(-2.5f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
model = glm::rotate(model, 200 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
FuegoTexture.UseTexture();
Material_opaco.UseMaterial(UniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

```

```

//cabeza 2
model = modelaux;
model = glm::translate(model, glm::vec3(-2.3f, 3.77f, -0.021f));
model = glm::rotate(model, 10*sin(glm::radians(varia2)) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 10* cos(glm::radians(varia2)) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
modelaux2 = model;
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(UniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));

```

```

Caardex_M.RenderModel();
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
//model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
RayosTexture.UseTexture();
Material_opaco.UseMaterial(UniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

```

```

//cabeza 3
model = modelaux;
model = glm::translate(model, glm::vec3(-2.364f, 2.71f, 0.53f));
model = glm::rotate(model, 10 * sin(glm::radians(2 * varia3)) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 10 * cos(glm::radians(2 * varia3)) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 10 * cos(glm::radians(2 * varia3)) * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux2 = model;
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(UniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
Caabizq_M.RenderModel();

```

```

//Dragon
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0 + ejexvaria, 5.0f * (5 * sin(glm::radians(angulovaria))), 6.0));
model = glm::scale(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::rotate(model, 10 * sin(glm::radians(varia2)) * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotDragon * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));

```

```

modelaux = model;
Material_brillante.UseMaterial(UniformSpecularIntensity, uniformShininess);
//color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(UniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cuerpo_M.RenderModel();

model = modelaux;
model = glm::translate(model, glm::vec3(-0.55f, 3.259f, -0.495f));
model = glm::rotate(model, 20*sin(glm::radians(varial)) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
Alader_M.RenderModel();

```

```

model = modelaux;
model = glm::translate(model, glm::vec3(-0.55f, 3.259f, 0.139f));
model = glm::rotate(model, 20*sin(glm::radians(varial)) * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
Alaizq_M.RenderModel();

```

```

//cabeza 1
model = modelaux;
model = glm::translate(model, glm::vec3(-2.05f, 3.85f, 0.45f));
model = glm::rotate(model, (14 * sin(glm::radians(varial)) * cos(glm::radians(varial))) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux2 = model;
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(UniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(UniformModel, 1, GL_FALSE, glm::value_ptr(model));
Caarizq_M.RenderModel();

```

```

model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
//model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
GasTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

//cabeza 4
model = modelaux;
model = glm::translate(model, glm::vec3(-2.42f, 2.69f, -1.035f));
model = glm::rotate(model, (4 * sin(glm::radians(2 * varia4))) * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux2 = model;
color = glm::vec3(0.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Caabder_M.RenderModel();

model = modelaux2;
model = glm::translate(model, glm::vec3(-3.7f, 0.3f, -1.4f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
HieloTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

//cabeza 5
model = modelaux;
model = glm::translate(model, glm::vec3(-2.55f, 3.029f, -0.18f));
model = glm::rotate(model, (7 * cos(glm::radians(2 * varia4))) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux2 = model;
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Caacetro_M.RenderModel();

model = modelaux2;
model = glm::translate(model, glm::vec3(-3.7f, 0.3f, -0.5f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//blending: transparencia o traslucidez
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
ArenaTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[3]->RenderMesh();

```



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

- En el primer ejercicio del cartel tuve problemas ya que no supe como seleccionar un rango en la imagen y mostrarlo en el cartel.
- Otro problema presentado fue el agregar las texturas a los elementos que según despedían las cabezas (fuego, rayos, gas, etc), lo cual por falta de tiempo no pude completar.
- Finalmente, otro problema fue el encontrar funciones adecuadas para el movimiento de las cabezas, e implementarlas correctamente.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
Los ejercicios para esta práctica fueron algo laboriosos y un tanto complicados el realizarlos, sobre todo para la parte del cartel con las letras.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
Se me dificulta un poco al momento de darle animación a los modelos para que sigan el recorrido que quiero que hagan, así como el tema de las rotaciones y el rango para ver lo de las letras del cartel.
- c. Conclusión
Me gusto la practica, aunque si se me hizo un poco difícil realizarla, aunque me parece muy bien que con el avance de las practicas vayamos haciendo cosas que debemos implementar en el proyecto final.